- **Deep explanation (interviewer-level)**
- **Types**
- **Properties**
- **Key differences**
- **Examples**
- **Interview insights**

🔷 Topic 1: CDI (Cloud Data Integration) – Deep Dive

1. What is CDI in Informatica IDMC?

Answer (interviewer-grade):
Cloud Data Integration (CDI) is Informatica's **cloud-native ETL/ELT engine** used to design, execute, and orchestrate data pipelines across **cloud and on-prem systems**.

It enables:

- Data ingestion
- Transformation
- Data movement
- Data synchronization

Key terminology:

- ETL / ELT
- Secure Agent runtime
- Pushdown optimization
- Hybrid data integration

2. CDI Architecture (Very Important)

CDI architecture consists of:

1. Design Layer

- Mapping Designer
- Taskflow Designer
- UI-based development

2. Execution Layer

- **Secure Agent** executes jobs
- Handles data movement and transformations

3. Connectivity Layer

- Connections to sources/targets (DB, APIs, files)

3. What is Secure Agent?

Answer:
Secure Agent is a **lightweight runtime engine** installed on-prem or cloud that executes CDI jobs securely.

Key properties:

- Handles data processing
- Manages connectivity
- Supports hybrid integration

Example:
Extract data from on-prem Oracle → load to Snowflake.

4. Types of Tasks in CDI

1. Mapping Task

- Executes mappings with transformations

2. Synchronization Task

- Simple replication (no complex logic)

3. Replication Task

- Bulk data replication across systems

5. Types of Loads

1. Full Load

- Loads entire dataset

2. Incremental Load

- Loads only changed data

3. CDC (Change Data Capture)

- Captures inserts, updates, deletes

6. Key CDI Properties (Important for Interviews)

Execution Properties

- Pushdown optimization
- Partitioning
- Batch size

Runtime Properties

- Parameterization
- Pre/Post SQL

Performance Properties

- Parallel processing
- Commit intervals

7. What is Pushdown Optimization?

Answer:
Pushdown optimization shifts transformation logic from Informatica to the **database layer**, improving performance.

Types:

1. **Source Pushdown** → executed at source
2. **Target Pushdown** → executed at target
3. **Full Pushdown** → entire logic executed in DB

8. What is Partitioning?

Answer:
Partitioning splits data into multiple streams for parallel processing.

Types:

- Hash partition
- Key range partition
- Round-robin

Example:
Process 1M rows using 4 parallel threads.

9. What is Parameterization?

Answer:
Using parameters and variables to make mappings reusable and dynamic.

Example:

- Source table name changes dynamically
- File path passed at runtime

10. What is Data Preview?

Answer:
Allows viewing sample data before execution to validate logic.

11. What is Mapping vs Mapping Task?

Difference (very important)

Feature	Mapping	Mapping Task
Purpose	Defines logic	Executes logic
Type	Design object	Runtime object
Execution	Cannot run directly	Can be scheduled/run

12. What is Synchronization Task vs Mapping Task?

Feature	Synchronization Task	Mapping Task
Complexity	Simple	Complex
Transformations	No	Yes
Use case	Data replication	ETL logic

13. What is Data Lineage in CDI?

Tracks data movement from **source → transformation → target**, useful for governance.

14. What is Data Integration Service?

Executes mappings and manages data pipelines inside Secure Agent.

15. What is Hybrid Integration?

Integration between:

- On-prem systems
- Cloud systems

Example:
Oracle (on-prem) → Snowflake (cloud)

🧠 Interview Insights for CDI

🔥 Common Interview Question:

👉 *"How does CDI execute a mapping?"*

Strong answer:

> Mapping is designed in UI → Mapping Task triggers execution → Secure Agent processes data → applies transformations → loads to target.

⚠️ Common Mistake:

Weak answer:

> "CDI moves data"

Strong answer:

> "CDI performs ETL/ELT processing using Secure Agent with support for pushdown optimization and hybrid connectivity."

💡 Key Differentiator

Strong candidates mention:

- Secure Agent role
- Pushdown optimization
- Partitioning for performance

✅ Summary

- CDI = core ETL engine
- Secure Agent = execution layer
- Pushdown = performance booster
- Tasks = execution control

Topic 2: Transformations in CDI (Deep Dive – Interview Level)

This is the **most important section in Informatica interviews**. Expect deep questions on:

- Types
- Behavior
- Performance
- Differences

1. What is a Transformation?

Answer (interviewer-grade):
A transformation is a **data processing component** in a mapping that performs operations such as filtering, joining, aggregating, or modifying data before loading it into the target.

Key terminology:

- Row-level vs set-level processing
- Data pipeline
- Active vs passive behavior

2. Types of Transformations

🔹 A. Based on Behavior

1. Active Transformation

- Changes number of rows (increase/decrease)
- Can filter, aggregate, or split data

Examples:

- Filter
- Aggregator
- Joiner

2. Passive Transformation

- Does NOT change row count
- Processes data row-by-row

Examples:

- Expression
- Lookup (non-filtering)

🔹 B. Based on Processing

1. Connected Transformation

- Directly part of data flow
- Receives and passes data automatically

2. Unconnected Transformation

- Called explicitly using functions
- Does not participate in pipeline

Example:
Unconnected Lookup

3. Common Transformations (Core Interview Focus)

3.1 Expression Transformation

Answer:
Performs row-level calculations and data manipulation.

Properties:

- Can create derived columns
- Supports conditional logic

Example:

```
salary * 1.1
```

Interview insight:
Used in almost every mapping.

3.2 Filter Transformation

Answer:
Filters rows based on a condition.

Properties:

- Drops rows not meeting condition
- Active transformation

Example:

```
salary > 50000
```

3.3 Router Transformation

Answer:
Routes data into multiple output groups based on conditions.

Difference vs Filter:

Feature	Filter	Router
Output	Single	Multiple
Use case	Simple filtering	Multi-condition routing

3.4 Aggregator Transformation

Answer (important):
Performs group-level calculations like SUM, AVG, COUNT.

Properties:

- Blocking transformation
- Requires grouping

Example:
Total sales per region

3.5 Joiner Transformation

Answer:
Joins data from multiple sources.

Types of Joins:

- Inner
- Left outer
- Right outer
- Full outer

Key terminology:

- Master vs Detail
- Join condition

3.6 Lookup Transformation

Answer:
Fetches data from a reference table.

Types of Lookup

1. Connected Lookup

- Part of data flow
- Returns multiple columns

2. Unconnected Lookup

- Called using function
- Returns single value

3. Static vs Dynamic Lookup

Type	Behavior
Static	Read-only
Dynamic	Updates lookup table

3.7 Sequence Generator

Answer:
Generates unique numeric values.

Use case:
Surrogate key generation

3.8 Rank Transformation

Answer:
Selects top or bottom N records.

Example:
Top 10 customers by sales

3.9 Sorter Transformation

Answer:
Sorts data based on specified columns.

Properties:

- Can remove duplicates
- Improves join performance

3.10 Union Transformation

Answer:
Combines multiple data streams into one.

3.11 Update Strategy

Answer:
Defines how data is written to target:

- Insert
- Update
- Delete

3.12 Normalizer

Answer:
Converts denormalized data into normalized form.

Example:
Repeating columns → rows

3.13 Transaction Control

Answer:
Controls commit/rollback logic.

3.14 SQL Transformation

Answer:
Executes SQL queries directly on database.

Key advantage:

- Pushdown optimization

3.15 Java Transformation

Answer:
Allows custom logic using Java code.

4. Blocking vs Non-Blocking Transformations

Blocking Transformation

- Waits for all data before processing

Examples:

- Aggregator
- Sorter

Non-Blocking Transformation

- Processes data row-by-row

Examples:

- Expression
- Filter

5. Key Differences (Very Important)

Active vs Passive

Feature	Active	Passive
Row count	Changes	Same
Example	Filter	Expression

Connected vs Unconnected Lookup

Feature	Connected	Unconnected
Data flow	Yes	No
Return	Multiple columns	Single value

Filter vs Router

Feature	Filter	Router
Outputs	One	Multiple
Complexity	Simple	Complex

6. Transformation Properties (Important)

Performance Properties

- Cache size (Lookup, Aggregator)
- Sorted input option

General Properties

- Input/output ports
- Data types

Advanced Properties

- Null handling
- Error handling

7. Real Interview Scenarios

🔥 Scenario 1:

👉 *"How do you remove duplicates?"*

Answer:

- Use Sorter (distinct option)
- Or Aggregator with group by

🔥 Scenario 2:

👉 *"When to use Joiner vs Source Qualifier?"*

Answer:

- Use Joiner → different sources
- Use Source Qualifier → same source

🔥 Scenario 3:

👉 *"How to generate surrogate key?"*

Answer:

- Use Sequence Generator

🧠 Interview Insights

What interviewer expects:

- Clear understanding of:
  - Active vs Passive
  - Blocking vs Non-blocking
  - Lookup types

Common mistake:

Weak:

> "Aggregator does sum"

Strong:

> "Aggregator is a blocking transformation that performs group-level calculations using cache."

✅ Summary

- Transformations = core of ETL logic
- Lookup, Joiner, Aggregator = most asked
- Active vs Passive = must know
- Blocking behavior impacts performance

Topic 3: Tasks & Taskflows in CDI (Deep Dive – Interview Level)

This topic tests your understanding of **execution, orchestration, and control flow** — very important for **real-time project scenarios**.

1. What is a Task in Informatica IDMC?

Answer (interviewer-grade):
A task is an **executable object** that runs data integration logic such as mappings, synchronization, or replication.

Key terminology:

- Runtime execution unit
- Parameter-driven execution
- Orchestration component

2. Types of Tasks in CDI

🔹 1. Mapping Task

Answer:
Executes a mapping with defined runtime properties such as source/target, parameters, and performance settings.

Properties:

- Pushdown optimization
- Partitioning
- Pre/Post SQL

🔹 2. Synchronization Task

Answer:
Used for simple data replication without complex transformations.

Use case:

- Copy table from source to target

🔹 3. Replication Task

Answer:
Used for large-scale data replication, often for cloud migration scenarios.

Key difference vs Sync Task:

- More optimized for bulk movement

3. What is a Taskflow?

Answer (very important):
A taskflow is a **workflow orchestration engine** used to control execution of multiple tasks in a sequence or parallel.

Key terminology:

- Workflow orchestration
- Control flow
- Dependency management

4. Types of Taskflows

🔹 1. Linear Taskflow

- Executes tasks **sequentially (one after another)**
- Simple and predictable

🔹 2. Non-Linear Taskflow

- Supports **branching, parallel execution, and conditions**
- Used for complex workflows

5. Taskflow Components (Core Interview Focus)

5.1 Start Task

Defines the starting point of a taskflow.

5.2 Mapping Task (inside Taskflow)

Executes mapping within workflow.

5.3 Decision Task

Answer:
Used to control flow based on conditions.

Example:
If file exists → run mapping, else → send alert

5.4 Assignment Task

Assigns values to variables.

5.5 Command Task

Executes OS-level commands.

Example:
Run shell script after job completion

5.6 Timer Task

Introduces delay in execution.

5.7 Event Task

Triggers execution based on events.

5.8 Email Task

Sends notifications on success/failure.

6. Execution Modes

Sequential Execution

- Tasks run one after another
- Used for dependency-based workflows

Parallel Execution

- Tasks run simultaneously
- Improves performance

7. Taskflow Properties (Important)

Runtime Properties

- Parameters
- Variables
- Input/output fields

Execution Properties

- Retry attempts
- Timeout settings

Scheduling Properties

- Cron-based scheduling
- Event-based triggers

8. Parameterization in Tasks

Answer:
Tasks support parameters and variables to make workflows **dynamic and reusable**.

Example:

- File path passed at runtime
- Date parameter for incremental load

9. What is Scheduling?

Answer:
Defines when a task or taskflow runs.

Types:

- Time-based (cron)
- Event-based

10. What is Error Handling in Taskflows?

Answer:
Handles failures using:

- Decision tasks
- Retry mechanisms
- Failure paths

Retry Mechanism

- Automatically retries failed tasks
- Configurable attempts

11. Logging and Monitoring

Logging

- Stores execution details
- Helps debugging

Monitoring

- Tracks execution status
- Shows success/failure

12. Mapping Task vs Taskflow (Very Important)

Feature	Mapping Task	Taskflow
Purpose	Execute mapping	Orchestrate multiple tasks
Complexity	Simple	Complex workflows
Control flow	No	Yes

13. Synchronization Task vs Replication Task

Feature	Sync Task	Replication Task
Use case	Simple copy	Large-scale replication
Performance	Moderate	Optimized

14. Real Interview Scenarios

🔥 Scenario 1:

👉 *"How do you build a workflow with dependency?"*

Answer:

Use Taskflow with sequential execution and decision tasks to control flow.

🔥 Scenario 2:

👉 *"How do you handle failure in workflow?"*

Answer:

Implement retry logic, use decision tasks, and configure failure paths with notifications.

🔥 Scenario 3:

👉 *"How to run tasks in parallel?"*

Answer:

Use non-linear taskflow with parallel branches.

🧠 Interview Insights

What interviewer expects:

- Clear difference:
  - Task vs Taskflow
  - Sequential vs Parallel

Common mistake:

Weak:

"Taskflow runs tasks"

Strong:

"Taskflow orchestrates execution using control flow constructs like decision, assignment, and parallel branches."

Key differentiator:

Mention:

- Retry logic
- Error handling
- Scheduling

✅ Summary

- Tasks = execution units
- Taskflows = orchestration engine
- Decision + Parallel = advanced workflows
- Error handling = critical in real projects

🔷 Topic 4: CAI (Cloud Application Integration) – Deep Dive (Interview Level)

This topic is crucial for **real-time integration, APIs, and event-driven architecture**. Interviewers expect you to think beyond ETL and explain **application-to-application integration**.

1. What is CAI in Informatica IDMC?

Answer (interviewer-grade):
Cloud Application Integration (CAI) is Informatica's **iPaaS (Integration Platform as a Service)** that enables **real-time integration between applications, APIs, and services**.

It supports:

- API-based integration
- Event-driven workflows
- Process orchestration

Key terminology:

- iPaaS
- Real-time integration
- Service orchestration

2. CAI Architecture

🔹 1. Process Designer

- UI to design workflows (processes)

🔹 2. Service Connectors

- Connect external systems (Salesforce, SAP, REST APIs)

🔹 3. API Manager

- Exposes and manages APIs

🔹 4. Runtime Engine

- Executes integration processes

3. What is a Process in CAI?

Answer:
A process is a **workflow that defines integration logic**, including data transformation, API calls, and orchestration.

Key terms:

- Orchestration
- Process flow

4. Types of Integration in CAI

🔹 1. Application Integration

- Connect apps like Salesforce, SAP

🔹 2. Data Integration

- Move and transform data

🔹 3. Process Integration

- Automate business workflows

🔹 4. B2B Integration

- Exchange data between organizations

5. What is an API in CAI?

Answer:
An API (Application Programming Interface) allows systems to **communicate and exchange data** using standard protocols.

Types of APIs

1. REST API

- Uses HTTP methods (GET, POST, PUT, DELETE)
- Lightweight and widely used

2. SOAP API

- Uses XML-based protocol
- More secure but heavier

6. What is API Manager?

Answer:
API Manager is used to create, publish, secure, and monitor APIs in CAI.

Key features:

- Rate limiting
- Authentication
- API versioning

7. What is a Service Connector?

Answer:
Service connectors are prebuilt components used to connect to external systems and APIs.

Examples:

- Salesforce connector
- REST connector

8. What is Event-driven Integration?

Answer:
Integration triggered automatically when an event occurs.

Key terminology:

- Event-based trigger
- Asynchronous processing

Example:
New order created → trigger process

9. What is Real-time Integration?

Answer:
Data is processed immediately as events occur, without batch delay.

Example:
API call updates customer data instantly.

10. What is a Webhook?

Answer:
A webhook is an **HTTP callback triggered by an event**, sending data to another system.

Example:
Payment success → webhook triggers order update.

11. What is Orchestration?

Answer:
Orchestration is the coordination of multiple services and APIs into a single workflow.

Key terms:

- Service coordination
- Workflow automation

12. What is Data Mapping in CAI?

Answer:
Mapping defines how data fields from source are transformed and mapped to target.

13. What is Error Handling in CAI?

Answer:
Handles failures using:

- Try-catch blocks
- Fault paths
- Retry mechanisms

14. What is API Gateway?

Answer:
A gateway that manages API traffic, security, and monitoring.

Key features:

- Authentication
- Throttling
- Logging

15. What is Synchronous vs Asynchronous Integration?

Synchronous

- Request → Response (real-time)

Asynchronous

- Event-based, no immediate response

16. Difference: CDI vs CAI (Very Important)

Feature	CDI	CAI
Type	Batch ETL	Real-time integration
Use case	Data pipelines	API integration
Processing	Batch	Event-driven

17. REST vs SOAP API

Feature	REST	SOAP
Format	JSON	XML
Speed	Faster	Slower
Complexity	Simple	Complex

18. Real Interview Scenarios

🔥 Scenario 1:

👉 *"How do you integrate Salesforce with SAP in real time?"*

Answer:

Use CAI with service connectors, REST APIs, and event-driven triggers for real-time synchronization.

🔥 Scenario 2:

👉 *"How do you expose a process as an API?"*

Answer:

Use API Manager to publish the process, apply security policies, and expose endpoints.

🔥 Scenario 3:

👉 *"When to use CAI vs CDI?"*

Answer:

Use CAI for real-time API-based integration and CDI for batch data processing.

🧠 Interview Insights

What interviewer expects:

- Clear understanding of:
  - API concepts
  - Event-driven architecture
  - Real-time vs batch

Common mistake:

Weak:

"CAI connects systems"

Strong:

"CAI enables real-time, event-driven integration using APIs and orchestration workflows."

Key differentiator:

Mention:

- Webhooks
- API Manager
- Orchestration

✅ Final Summary (IDMC Mastery)

CDI

- Batch ETL processing

Transformations

- Data logic engine

Tasks

- Execution & orchestration

CAI

- Real-time API integration

🔷 Topic 5: Scenario-Based & Advanced IDMC Questions (CDI + CAI Combined)

This is where interviews become **practical**. You'll be judged on:

- Design thinking
- Tool selection (CDI vs CAI)
- Performance + error handling

1. End-to-End ETL + Real-Time Architecture

Q1. How would you design an end-to-end pipeline from on-prem Oracle to Snowflake?

Answer (strong interview version):

I would use **CDI for batch ingestion** from Oracle using Secure Agent, apply transformations (Expression, Lookup, Aggregator), and load into Snowflake. For real-time updates, I would use **CAI APIs or CDC-based pipelines**.

Architecture flow:

- Oracle → Secure Agent → Mapping → Snowflake

Key terms:

- Hybrid integration
- Batch + real-time combination

Q2. How do you handle incremental loading?

Answer:

Use **CDC (Change Data Capture)** or filter based on **last updated timestamp**, combined with parameterization for dynamic execution.

Example:

- WHERE last_updated > $$last_run_time

Q3. How would you design a real-time integration between two applications?

Answer:

Use CAI with REST APIs, event triggers, and process orchestration. Webhooks can be used to trigger workflows instantly.

2. Performance Optimization Scenarios

Q4. Your mapping is slow — how do you optimize?

Answer (structured):

1. Enable **pushdown optimization**
2. Use **partitioning**
3. Optimize transformations (avoid unnecessary joins)
4. Tune **lookup caching**
5. Reduce data volume early (filter)

Q5. When would you use pushdown optimization?

Answer:

When source/target databases can handle transformations efficiently, reducing load on Secure Agent.

Q6. How do you optimize lookup performance?

Answer:

- Use **static lookup cache**
- Enable indexing on lookup table
- Reduce lookup columns

3. Transformation-Based Scenarios

Q7. How do you implement SCD Type 2?

Answer:

Use Lookup to check existing records, Sequence Generator for surrogate key, and Update Strategy to insert new versions with effective dates.

Q8. How do you remove duplicates?

Answer:

- Sorter (distinct option)
- Aggregator (group by key fields)

Q9. Joiner vs Lookup — when to use?

Scenario	Use
Large dataset join	Joiner
Small reference lookup	Lookup

4. Taskflow & Workflow Scenarios

Q10. How do you build dependency between tasks?

Answer:

Use Taskflow with **sequential execution** and decision tasks to control flow.

Q11. How do you handle task failure?

Answer:

- Retry logic
- Decision task for alternate path
- Email notification

Q12. How to run tasks in parallel?

Answer:

Use **non-linear taskflow with parallel branches**

5. CAI (API & Real-Time) Scenarios

Q13. How do you expose an ETL process as API?

Answer:

Use CAI Process Designer → create process → publish via API Manager → secure using authentication.

Q14. How do you handle API failures?

Answer:

- Retry mechanism
- Exception handling
- Logging

Q15. How do you integrate Salesforce with Snowflake?

Answer:

- Use CAI for real-time sync
- Use CDI for batch data loads

6. Design & Decision Questions (VERY IMPORTANT)

Q16. When to use CDI vs CAI?

Use Case	Tool
Batch ETL	CDI
Real-time API	CAI
Hybrid	Both

Q17. How do you design a scalable pipeline?

Answer:

Use partitioning, pushdown optimization, parallel execution, and modular mappings.

Q18. How do you ensure data quality?

Answer:

- Validation rules
- Error handling
- Data profiling

Q19. How do you handle large data volumes?

Answer:

- Partitioning
- Pushdown optimization
- Bulk processing

Q20. How do you secure data pipelines?

Answer:

- Role-based access
- Encryption
- Secure Agent

7. Advanced Real Interview Questions

Q21. How do you design fault-tolerant pipelines?

Answer:

Use retry mechanisms, checkpoints, logging, and failure paths in taskflows.

Q22. How do you implement audit logging?

Answer:

Maintain audit tables with:

- Start time
- End time
- Status
- Row counts

Q23. How do you manage schema changes?

Answer:

Use dynamic mappings and metadata-driven design.

Q24. How do you handle API rate limits?

Answer:

Use throttling, batching, and retry strategies.

Q25. How do you debug a failed job?

Answer:

Analyze logs, check query execution, validate mappings, and review error messages.

🧠 Final Interview Master Insight

🔥 Most Powerful Answer Pattern

Whenever asked ANY scenario:

👉 Use this structure:

1. Understand requirement
2. Choose tool (CDI/CAI)
3. Design flow
4. Optimize performance
5. Handle errors

💡 Example Strong Answer:

I would design the pipeline using CDI for batch processing, apply transformations with pushdown optimization, orchestrate using taskflows, and use CAI for real-time integration where required.

✅ You Are Now Interview-Ready For:

- Informatica IDMC (CDI + CAI)
- Snowflake integration
- Real-time + batch pipelines

🔷 Pushdown Optimization & Performance Tuning in Informatica CDI (Deep Interview Guide)

This is a **high-weight interview topic** — especially for **2+ years experience**. Interviewers expect:

- Concept clarity
- When to use vs not use
- Real-world optimization strategy

🔹 1. What is Pushdown Optimization?

Answer (interviewer-grade):
Pushdown Optimization is a technique where **transformation logic is pushed from Informatica Secure Agent to the database**, allowing the database engine to execute SQL operations instead of the ETL engine.

Key terminology:

- Query pushdown
- In-database processing
- Reduced data movement

🔷 2. Why Pushdown Optimization?

Core idea:

> "Move computation closer to data"

Benefits:

- Improves performance
- Reduces network I/O
- Leverages database power (indexes, parallelism)

🔷 3. Types of Pushdown Optimization

🔸 1. Source Pushdown

- Logic executed at **source database**
- Only required data is extracted

Example:

```
SELECT * FROM sales WHERE amount > 1000;
```

🔸 2. Target Pushdown

- Logic executed at **target database**
- Data is loaded first, then processed

🔸 3. Full Pushdown (Most Important)

- Entire transformation logic executed in database
- Informatica acts only as orchestrator

🔷 4. Pushdown Optimization Levels

Level	Description
None	All processing in Secure Agent
Partial	Some transformations pushed
Full	Entire logic pushed

🔷 5. When to Use Pushdown Optimization?

✅ Use When:

- Source/target is powerful DB (Snowflake, Oracle)
- Large data volumes
- Simple transformations (filter, join, aggregation)

❌ Avoid When:

- Complex transformations (Java, custom logic)
- Unsupported transformations
- Source DB performance is poor

🔷 6. Transformations Supported in Pushdown

Common Supported:

- Filter
- Joiner
- Aggregator
- Expression (simple logic)

Not Fully Supported:

- Java Transformation
- Complex Lookup
- Custom logic

🔷 7. Pushdown vs Non-Pushdown

Feature	Pushdown	Non-Pushdown
Execution	Database	Secure Agent
Performance	Faster	Slower
Data movement	Less	More
Control	Less flexible	More flexible

🔷 8. Real Interview Scenario

👉 *"Your mapping is slow — what will you do?"*

Strong answer:

> First, I would enable pushdown optimization to shift processing to the database. Then I would analyze query performance, reduce data movement, and optimize transformations.

🔷 9. Performance Tuning in CDI (Full Strategy)

🧠 1. Source-Level Optimization

Techniques:

- Apply filters early
- Use indexed columns
- Reduce data extraction

Example:

Instead of loading full table → filter at source

🧠 2. Mapping-Level Optimization

Key Techniques:

🔸 Use Filter Early

- Reduce data volume ASAP

🔸 Optimize Transformations

- Avoid unnecessary Joiners
- Use Lookup instead of Join when possible

🔸 Use Expression Efficiently

- Avoid complex nested logic

🔸 **Reduce Data Type Conversion**

- Minimize casting

🧠 3. Lookup Optimization

Techniques:

- Use **static cache**
- Reduce lookup columns
- Use indexed lookup table

🧠 4. Partitioning (Very Important)

What is Partitioning?

Splitting data into multiple streams for parallel processing.

Types:

- Hash partition
- Round-robin
- Key range

Benefit:

- Improves throughput
- Parallel execution

🧠 5. Pushdown Optimization (Again Critical)

- Shift workload to DB
- Reduce Secure Agent load

🧠 6. Target-Level Optimization

Techniques:

- Bulk loading
- Disable indexes during load

- Use staging tables

🧠 7. Task-Level Optimization

Techniques:

- Increase commit interval
- Enable parallel execution
- Optimize batch size

🧠 8. Network Optimization

Techniques:

- Reduce data transfer
- Use compression
- Optimize connectivity

🧠 9. Secure Agent Optimization

Techniques:

- Increase memory
- Use multiple agents
- Monitor CPU usage

🔷 10. Common Performance Bottlenecks

Issues:

- Large data scans
- Poor join conditions
- No filtering
- Heavy transformations

🔷 11. End-to-End Performance Strategy (Best Answer)

👉 If interviewer asks:

"How do you tune performance in Informatica?"

Strong structured answer:

> I start by optimizing at the source by filtering data early, then optimize mappings using efficient transformations and lookup caching. I enable pushdown optimization where possible, apply partitioning for parallelism, and finally tune task-level properties like commit intervals and batch size.

🔷 12. Pushdown + Partitioning (Advanced Insight)

👉 Strong candidates say:

> Combining pushdown optimization with partitioning leverages both database parallelism and ETL parallelism, giving maximum performance.

🔷 13. Real Interview Trick Questions

Q: Why is pushdown not always used?

Answer:

> Because not all transformations are supported, and sometimes database performance may become a bottleneck.

Q: Which is better — pushdown or partitioning?

Answer:

> Both serve different purposes. Pushdown reduces data movement, while partitioning increases parallel processing. Best performance comes from combining both.

✅ Final Summary

Pushdown Optimization

- Move logic to DB
- Reduce data movement
- Improve speed

Performance Tuning

- Filter early
- Optimize transformations
- Use partitioning
- Tune tasks

🧠 Final Interview Tip

👉 Always answer like this:

> "I optimize performance at multiple layers — source, mapping, and task level — using pushdown optimization, partitioning, and efficient transformation design."

🔥 Top Tricky Interview Questions (IDMC)

1. Why is your mapping slow even after increasing resources?

What interviewer tests: Understanding beyond brute-force scaling

Strong Answer:

> Increasing resources doesn't always help if the issue is poor design. I would check query profile, ensure early filtering, optimize joins/lookups, and enable pushdown optimization before scaling compute.

2. When should you NOT use Pushdown Optimization?

Testing: Practical understanding

Answer:

> Avoid pushdown when transformations are unsupported, logic is complex (Java/custom), or the source system cannot handle additional load.

3. Lookup vs Joiner — which is faster?

Testing: Performance judgment

Answer:

Lookup is generally faster for small reference data due to caching, while Joiner is better for large datasets or when joining multiple large sources.

4. Why is your Lookup slow?

Testing: Optimization skills

Answer:

Possible reasons include large lookup table, no indexing, dynamic cache overhead, or excessive columns. I would optimize cache, reduce columns, and ensure indexing.

5. How do you handle duplicate data in high-volume pipelines?

Testing: Real-world problem solving

Answer:

Use Sorter (distinct), Aggregator (group by), or deduplication logic at source. Prefer handling duplicates early to reduce processing load.

6. Why is Aggregator considered expensive?

Testing: Internal behavior knowledge

Answer:

Aggregator is a blocking transformation — it needs to read all rows before processing, which increases memory usage and latency.

7. How do you optimize a Joiner transformation?

Testing: Deep tuning

Answer:

Use sorted input, choose smaller dataset as master, reduce columns, and push join logic to database using pushdown optimization.

8. What happens if Secure Agent goes down?

Testing: Architecture understanding

Answer:

Jobs fail since execution depends on Secure Agent. For high availability, we deploy multiple agents or agent groups.

9. Partitioning vs Pushdown — which is better?

Testing: Concept clarity

Answer:

They serve different purposes. Pushdown reduces data movement, while partitioning increases parallelism. Best performance comes from combining both.

10. Why is filtering early important?

Testing: Data pipeline thinking

Answer:

Filtering early reduces data volume, which minimizes processing, memory usage, and network overhead.

11. Why is your Taskflow failing intermittently?

Testing: Debugging mindset

Answer:

Could be due to network issues, API timeouts, resource limits, or data inconsistencies. I would check logs, retries, and external dependencies.

12. When do you use Dynamic Lookup?

Testing: Advanced transformation knowledge

Answer:

When lookup table needs to be updated during processing, such as maintaining dimension tables in SCD scenarios.

13. Why is your pushdown optimization not working?

Testing: Troubleshooting

Answer:

Could be due to unsupported transformations, incompatible databases, or incorrect configuration. I would verify pushdown settings and mapping compatibility.

14. What is the biggest performance bottleneck in ETL?

Testing: System thinking

Answer:

Data movement and I/O operations are the biggest bottlenecks, not computation.

15. Why use Lookup instead of Joiner?

Testing: Decision-making

Answer:

Lookup is preferred for small reference data due to caching, while Joiner is better for large dataset joins.

16. How do you handle API rate limits in CAI?

Testing: Real-time integration

Answer:

Implement throttling, batching, and retry mechanisms with exponential backoff.

17. What happens if data changes during incremental load?

Testing: Data consistency

Answer:

Use CDC or timestamp-based logic to ensure only committed changes are processed.

18. Why is your mapping consuming high memory?

Testing: Internal behavior

Answer:

Likely due to blocking transformations like Aggregator or large lookup cache. I would optimize cache size and reduce data volume.

19. How do you design fault-tolerant pipelines?

Testing: Production readiness

Answer:

Use retry mechanisms, checkpoints, logging, error handling paths, and idempotent processing.

20. Why use Taskflow instead of multiple Mapping Tasks?

Testing: Architecture thinking

Answer:

Taskflow provides orchestration, dependency management, error handling, and parallel execution.

21. How do you debug a failed mapping?

Testing: Practical skills

Answer:

Check logs, validate source/target, review transformation logic, and use data preview.

22. Why is sorting required before Joiner?

Testing: Optimization

Answer:

Sorted input improves join performance and reduces memory usage.

23. What is the impact of data skew?

Testing: Advanced performance

Answer:

Uneven data distribution leads to imbalance in processing, causing slower execution.

24. Why is real-time integration harder than batch?

Testing: CAI understanding

Answer:

Real-time requires low latency, error handling, and event-driven architecture, unlike batch which is simpler and scheduled.

25. If database is slow, what will you do?

Testing: Balanced thinking

Answer:

Avoid overloading DB with pushdown, optimize queries, and shift processing back to ETL layer if needed.

🧠 Final Interview Strategy

🔥 Golden Rule

👉 Always answer like:

> "I would first analyze the root cause, then optimize at source, mapping, and execution levels."

💡 What makes you stand out:

- You **don't jump to scaling**
- You **think in layers**:
  - Source
  - Mapping
  - Task
  - Infrastructure

⚠️ Most Important Insight

👉 Interviewers don't want definitions
👉 They want **decision-making + trade-offs**

🔷 Topic 6: Real Project-Based Scenarios (IDMC CDI + CAI)

These are **not theoretical** — these simulate **actual interview discussions**.

🏗️ Scenario 1: End-to-End Data Pipeline

Q1. Explain a project where you used Informatica IDMC.

What interviewer expects: Real architecture + clarity

Strong Answer (template):

> In my project, I designed an end-to-end data pipeline using CDI to extract data from an on-prem Oracle database via Secure Agent. I applied transformations like Expression, Lookup, and Aggregator, and loaded the data into Snowflake. For orchestration, I used Taskflows, and for real-time integration, I used CAI APIs.

🔑 Key Points to Mention:

- Source → Oracle

- Target → Snowflake
- Tool → CDI + CAI
- Runtime → Secure Agent

🏗️ Scenario 2: Incremental Load Design

Q2. How did you implement incremental loading?

Strong Answer:

> I implemented incremental load using a timestamp column (last_updated) and parameterized the mapping to process only new or updated records. In some cases, I used CDC for capturing inserts, updates, and deletes.

💡 Pro Tip:

Mention both:

- Timestamp-based
- CDC-based

🏗️ Scenario 3: Handling Large Data Volumes

Q3. How did you handle millions of records?

Strong Answer:

> I used partitioning to enable parallel processing, applied pushdown optimization to reduce data movement, and filtered data at the source level to minimize processing overhead.

🔑 Keywords:

- Partitioning
- Pushdown
- Early filtering

🏗️ Scenario 4: Performance Issue

Q4. Tell me about a performance issue you faced.

Strong Answer:

One mapping was slow due to large joins and unnecessary data processing. I optimized it by filtering data early, replacing Joiner with Lookup where applicable, enabling pushdown optimization, and increasing partitioning.

💡 Key Insight:

Always show:

- Problem
- Analysis
- Solution

🏗️ Scenario 5: Error Handling

Q5. How do you handle failures in pipelines?

Strong Answer:

I implemented error handling using taskflows with decision tasks, retry mechanisms, and email notifications. I also maintained audit logs to track failures.

🏗️ Scenario 6: Real-Time Integration (CAI)

Q6. How did you implement real-time integration?

Strong Answer:

I used CAI to create REST APIs and event-driven processes. Webhooks triggered workflows, and service connectors integrated systems like Salesforce and SAP.

🔑 Keywords:

- REST API
- Webhooks
- Event-driven

🏗️ Scenario 7: SCD Implementation

Q7. How did you implement SCD Type 2?

Strong Answer:

I used Lookup to check existing records, Sequence Generator for surrogate keys, and Update Strategy to insert new records with effective dates and maintain history.

🏗️ Scenario 8: Data Quality Issue

Q8. How do you ensure data quality?

Strong Answer:

I applied validation rules, handled null values, removed duplicates, and implemented error logging for invalid records.

🏗️ Scenario 9: Choosing CDI vs CAI

Q9. When did you use CDI vs CAI?

Strong Answer:

I used CDI for batch ETL pipelines and CAI for real-time API-based integrations. In some cases, I combined both for hybrid workflows.

🏗️ Scenario 10: Production Deployment

Q10. How do you deploy pipelines?

Strong Answer:

I use parameterization, environment variables, and scheduling. I also validate mappings, monitor execution, and maintain logs for production support.

🧠 Advanced Follow-Up Questions (Interview Trick Layer)

Q11. What challenges did you face?

Answer:

Performance issues, handling large data volumes, and ensuring real-time integration reliability.

Q12. How did you optimize cost?

Answer:

Reduced data movement, used pushdown optimization, and optimized scheduling.

Q13. How do you ensure scalability?

Answer:

By using partitioning, modular mappings, and scalable cloud architecture.

Q14. How do you monitor pipelines?

Answer:

Using logs, monitoring dashboards, and alerts.

Q15. How do you handle schema changes?

Answer:

By using dynamic mappings and metadata-driven design.

🧠 Final Interview Strategy (MOST IMPORTANT)

🔥 Golden Answer Structure

Whenever asked ANY project question:

👉 Follow this format:

1. **Requirement**
2. **Tool used (CDI/CAI)**
3. **Design approach**
4. **Performance optimization**
5. **Error handling**

💡 Example Final Answer

I designed the pipeline using CDI for batch processing, applied transformations with pushdown optimization, orchestrated using taskflows, and used CAI for real-time integrations. I ensured performance using partitioning and handled errors using retry and logging mechanisms.

Lookup Cache in Informatica CDI – Types, Usage, Examples (Interview Deep Dive)

Lookup cache is a **highly tested topic** in interviews — especially for **performance tuning + SCD scenarios**.

🧠 1. What is Lookup Cache?

Answer (interviewer-grade):
Lookup cache is a **memory structure used to store lookup table data** during execution, reducing repeated database access and improving performance.

Key terminology:

- In-memory caching
- Cache initialization
- Cache lookup

🔷 2. Types of Lookup Cache

🔹 1. Static Cache

✅ Definition:

Static cache is **loaded once at the beginning of the session** and remains unchanged throughout execution.

📌 Properties:

- Read-only cache
- No updates during execution
- Faster lookup after initial load

🎯 When to Use:

- Reference data does NOT change
- Small to medium lookup tables
- Dimension tables (stable data)

📌 Example:

👉 Lookup table: Country codes

IN → India

US → USA

This data rarely changes → use static cache

⚠️ Interview Insight:

Static cache is preferred for performance when data is stable.

🔷 2. Dynamic Cache (Most Important)

✅ Definition:

Dynamic cache is **updated during mapping execution**, allowing inserts/updates into lookup table.

📌 Properties:

- Cache is modified during execution
- Supports insert/update
- Used for SCD scenarios

🎯 When to Use:

- Lookup table changes during processing
- Slowly Changing Dimension (SCD Type 1 / Type 2)
- Deduplication logic

📌 Example:

👉 Customer table:

- New customer arrives → insert into lookup
- Existing customer → update record

⚠️ Interview Insight:

Dynamic cache enables incremental updates without reloading entire table.

🔷 3. Persistent Cache

✅ Definition:

Persistent cache is **stored on disk and reused across sessions**, avoiding reloading cache every time.

📌 Properties:

- Saved to disk
- Reused in next runs
- Faster initialization

🎯 When to Use:

- Large lookup tables
- Repeated executions
- Performance-critical pipelines

📌 Example:

👉 Lookup table with 10M records

Instead of loading every time → reuse cache

⚠️ Interview Insight:

Reduces startup time significantly.

🔷 4. Shared Cache

✅ Definition:

Shared cache allows **multiple transformations or mappings to reuse the same cache**.

📌 Properties:

- Reusable across lookups
- Reduces memory usage
- Improves performance

🎯 When to Use:

- Multiple lookups using same data

- Complex mappings with repeated lookups

📌 Example:

👉 Customer lookup used in 3 places

Share cache instead of creating 3 caches

🔷 3. Summary Table (Very Important)

Cache Type	Updates During Run	Stored on Disk	Use Case
Static	❌ No	❌ No	Stable reference data
Dynamic	✅ Yes	❌ No	SCD, incremental updates
Persistent	❌ No	✅ Yes	Large datasets, repeated runs
Shared	Depends	Depends	Multiple lookups

🔷 4. Key Differences (Interview Focus)

Static vs Dynamic Cache

Feature	Static	Dynamic
Updates	No	Yes
Use case	Reference data	SCD / incremental
Performance	Faster	Slightly slower

Dynamic vs Persistent Cache

Feature	Dynamic	Persistent
Update	Yes	No
Storage	Memory	Disk
Use case	Real-time update	Reuse cache

🔷 5. Real Interview Scenarios

🔥 Q1: When would you use Dynamic Lookup?

Answer:

When the lookup table needs to be updated during processing, such as SCD Type 2 implementation.

🔥 Q2: Why is Static Cache faster?

Answer:

Because it is loaded once and not modified, avoiding additional overhead.

🔥 Q3: When to use Persistent Cache?

Answer:

When lookup table is large and reused across multiple runs to avoid reloading overhead.

🔥 Q4: What happens if cache is too large?

Answer:

It may cause memory issues. In such cases, reduce columns, use filtering, or switch to persistent cache.

🔷 6. Performance Tips (Very Important)

✅ Best Practices:

- Use **Static Cache** for stable data
- Use **Dynamic Cache** only when needed
- Reduce lookup columns
- Apply filters before lookup
- Use indexed lookup tables

⚠️ Avoid:

- Large dynamic cache unnecessarily
- Lookup on unfiltered large dataset
- Repeated lookup without shared cache

🧠 Final Interview Insight

👉 Strong candidates say:

> "I choose lookup cache type based on data behavior — static for stable data, dynamic for incremental updates, and persistent for large reusable datasets."

✅ Final Takeaway

- Static → Fast, read-only
- Dynamic → Flexible, update
- Persistent → Reusable
- Shared → Efficient